class from listing 2.3 the destructor had to check that the thread was still joinable, you can do that in the constructor ❷ and throw an exception if it's not.

The move support in `std::thread` also allows for containers of `std::thread` objects, if those containers are move aware (like the updated `std::vector<>`). This means that you can write code like that in the following listing, which spawns a number of threads and then waits for them to finish.

**Listing 2.7. Spawn some threads and wait for them to finish**

```
void do_work(unsigned id);

void f()
{
    std::vector<std::thread> threads;
    for(unsigned i=0;i<20;++i)
    {
        threads.push_back(std::thread(do_work,i));
    }
    std::for_each(threads.begin(),threads.end(),
                  std::mem_fn(&std::thread::join));
}
```

Spawn threads

Call join() on each thread in turn

If the threads are being used to subdivide the work of an algorithm, this is often just what's required; before returning to the caller, all threads must have finished. Of course, the simple structure of listing 2.7 implies that the work done by the threads is self-contained, and the result of their operations is purely the side effects on shared data. If `f()` were to return a value to the caller that depended on the results of the operations performed by these threads, then as written this return value would have to be determined by examining the shared data after the threads had terminated. Alternative schemes for transferring the results of operations between threads are discussed in chapter 4.

Putting `std::thread` objects in a `std::vector` is a step toward automating the management of those threads: rather than creating separate variables for those threads and joining with them directly, they can be treated as a group. You can take this a step further by creating a dynamic